INF3880 – Inteligencia Artificial Agéntica

Tarea 1 · Asistente de Personas con tools (20 %)

Publicación: miércoles 04/11/2026 (clase 3)

Entrega: martes 17/11/2026 hasta las 23:59 (clase 5)

Modalidad: individual o en parejas

Resultados de aprendizaje: RA 2 (diseñar prompts y tools para un agente) y RA 3 (implementar y evaluar un loop agéntico)

---

1. Contexto

Lumen Retail es una cadena chilena de tiendas (ficticia). Su área de Personas recibe todos los días las mismas preguntas por correo y chat: "¿cuántos días de vacaciones me quedan?", "¿en qué está mi solicitud?", "¿con cuánta anticipación tengo que pedir vacaciones?", "¿puedo trabajar remoto la próxima semana?". Quieren un **asistente interno** que responda a colaboradores y jefaturas usando los datos de sus sistemas, que **nunca invente** información y que **derive a una persona** cuando corresponda.

Los sistemas están simulados con archivos JSON en `datos/`:

Archivo	Contenido
`colaboradores.json`	9 personas: id, nombre, cargo, área, jefatura, fecha de ingreso, modalidad, saldo de vacaciones
`solicitudes.json`	9 solicitudes de vacaciones, permisos, licencias y teletrabajo con su estado
`politicas.json`	7 políticas internas (vacaciones, permisos, licencias, teletrabajo, jornada, beneficios, privacidad)
`feriados.json`	feriados de referencia para contar días hábiles

Lee los datos antes de empezar: tienen casos pensados para hacerte pensar (dos personas llamadas Camila, una colaboradora con menos de un año, una solicitud rechazada, una política de privacidad que limita qué se puede responder a quién).

2. Lo que ya está resuelto (código base)

Archivo	Qué hace	¿Se modifica?
`cliente.py`	Cliente de Gemini con tu key gratuita (desde `.env`), elección automática de modelo con respaldo, reintentos ante errores 429/500, logging a `logs/agente.log`, estimación de costo	No
`tools.py`	Carga de datos; **una tool implementada** (`buscar_colaborador`) con su declaración; cuatro tools con la firma, el docstring y la declaración a completar	Sí (las secciones con comentario TODO)
`agent.py`	Esqueleto del loop agéntico con la llamada al modelo ya escrita; `ResumenConsulta` (Pydantic) esbozado	Sí (las secciones con comentario TODO)
`run_eval.py`	Ejecuta un archivo de casos, imprime una tabla y guarda `resultados/*.jsonl`	No
`tests/`	Tests de las tools (sin red) y del loop (con un modelo falso)	No (pero debes agregar tests propios, ver sección 3)

Instalación y flujo de trabajo sugerido: ver `README.md`.

3. Lo que debes implementar

1. **System prompt** (`agent.py`, TODO 1). Rol, a quién atiende, reglas y formato. Debe cubrir al menos: no inventar datos y usar siempre las tools; pedir aclaración cuando falte información o haya ambigüedad (por ejemplo, dos coincidencias de nombre); qué hacer cuando una tool falla; y la **política de privacidad** (POL-07): cada consulta llega precedida por una línea `[Consulta de: <id> <nombre>, <cargo>]` que

identifica a quien pregunta, y el asistente solo puede entregar información individual del propio consultante o, si es jefatura directa, de su equipo.

2. **Loop agéntico completo** (`agent.py`, TODO 2–4), con el mismo patrón del Práctico 3: detectar `function_calls`, ejecutar la tool, devolver el resultado con `Part.from_function_response`, repetir hasta la respuesta final; **límite de iteraciones** con una respuesta controlada al cortarse; **manejo de errores de tools** (una excepción en una tool no debe matar el loop: se convierte en un resultado `{"error": ...}` que el modelo explica).
3. **Al menos tres tools nuevas** (`tools.py`), de las cuatro propuestas: `saldo_vacaciones`, `estado_solicitudes`, `buscar_politica`, `dias_habiles_entre`. Cada una con su función y su `FunctionDeclaration` (nombre, **descripción que diga cuándo usarla**, parámetros en JSON Schema). Pueden proponer una quinta tool propia si la justifican.
4. **Salida estructurada** (`agent.py`, TODO 5): `resumir()` debe devolver un `ResumenConsulta` validado con Pydantic usando `response_schema`, como en el Práctico 2. Pueden ajustar los campos; expliquen en el informe para qué los usaría Personas.
5. **Set de 10 casos** (`casos/casos.json`), partiendo de los 3 de ejemplo, que incluya al menos: 3 normales, 1 que **encadene dos tools**, 1 **ambiguo** (nombre repetido o información faltante), 1 **sin datos** (persona o política inexistente), 1 de **política** (pregunta sobre una regla, no sobre datos), 1 de **privacidad** (alguien pregunta por un tercero que no es de su equipo) y 1 que **provoque un error de tool** (por ejemplo, una fecha mal formada). Para cada caso, el campo `esperado_contiene` es orientativo; el análisis real lo hacen leyendo la traza.
6. Test propios (tests/): agrega tests para tus tools nuevas (al menos un caso normal y uno de error por tool). Deben correr sin red.

4. Entregables

Un archivo `.zip`, subido a Canvas, con:

1. **Código** completo, sin la key (`.env` no se incluye; revisa que no quede en ningún archivo).
2. **Informe** de máximo 3 páginas siguiendo `informe_plantilla.md`: decisiones del system prompt y de las tools, con al menos un ejemplo antes/después de un cambio (en el prompt o en la descripción de una tool) y su efecto; resultados de los 10 casos y diferencias observadas entre corridas; al menos 2 fallas analizadas; costo estimado y declaración de uso de IA.
3. **Evidencia de ejecución:** los archivos `resultados/eval_<fecha>.jsonl` de al menos dos corridas completas de los 10 casos, y `logs/agente.log`.

5. Rúbrica (ponderación de la rúbrica común del curso)

Criterio	Excelente (100 %)	Suficiente (60 %)	Insuficiente (20 %)
Funcionamiento (40%)	Loop correcto con límite y manejo de errores; ≥ 3 tools operativas y bien declaradas; salida estructurada validada; los 10 casos corren y los tests pasan	Loop funciona en casos normales pero falla en borde (error de tool, límite, ambigüedad) o faltan tools/salida estructurada	No corre, o el loop no ejecuta tools, o faltan requisitos centrales
Diseño y justificación (25%)	System prompt y descripciones de tools justificados con antes/después; privacidad y ambigüedad resueltas con criterio; alternativas y trade-offs	Decisiones descritas sin evidencia de iteración ni comparación	Sin justificación o prompt genérico
Evaluación (20%)	10 casos que cubren todos los tipos; fallas analizadas con causa y propuesta; se observa la variabilidad entre corridas	Casos básicos, poco análisis de fallas	Sin casos o sin análisis
Calidad de código e informe (15%)	Código claro, tests propios, informe conciso y completo, sin key en la entrega	Funcional pero desordenado o informe incompleto	Difícil de seguir, o la key quedó en el código

Una key de API incluida en la entrega descuenta el 100 % del criterio de calidad y obliga a revocarla.

6. Reglas

- **Uso de IA:** permitido para código auxiliar, depuración y redacción, **declarándolo** en el informe (qué herramienta, para qué). El loop, el system prompt y el diseño de tools deben poder explicarlos en una defensa breve de 5 minutos si se les pide.
- **API:** cada estudiante usa su **key gratuita personal** de Google AI Studio (la misma de los prácticos). El curso no entrega keys ni créditos. No pongan datos reales de personas en los prompts (los términos del nivel gratuito permiten a Google usarlos).
- **Límites del nivel gratuito:** `cliente.py` ya reintenta ante errores 429 y 500. Si una corrida se corta, espera un minuto y repite; no es necesario que los 10 casos corran en una sola ejecución.
- **Parejas:** En caso de realizar la evaluación en parejas, ambos integrantes deben poder explicar todo el código. La nota es compartida.
- **Atraso:** descuento de 1 punto por día de atraso, máximo 3 días.

7. Consultas

Por el foro del curso (preferido, para que todos vean la respuesta) o a la ayudante, Michelle Madrid. Las preguntas sobre el enunciado que cambien algo se publicarán como fe de erratas en el foro.